* key insight; each number in a vector is just a learned coordinate in some high-dimensional space. The individual numbers don't have human-readable meaning, its only the relationships between vectors (angles + distances) that carry meaning.

Say you train a model on text and it learns the embeddings for 3 words:

each number is just a coordinate the model figured out on its own during training.

"king" → [0.99, 0.02, 0.74, ...]

"queen" → [0.95, 0.88, 0.71, ...]

"apple" → [0.01, 0.03, 0.92, ...]

example for understanding vectors / embeddings

SO, individual components of an embedding vector are meaningless alone BUT, the geometry between vectors is where the meaning lies.

Matrix

$$\begin{bmatrix} 7 & 10 & 8 & \cdots \\ 4 & 3 & 9 & \cdots \\ 1 & 2 & 3 & \cdots \\ \cdots & \cdots & \cdots & \cdots \end{bmatrix}$$

n-dimensional

when a matrix multiplies a vector and transforms it, it is a learned projection. The model learns which matrix makes the downstream task work the best. The weights of neural networks are mostly just matrices being applied one after another, each reshaping the data to be in a more useful form.

example for understanding matrix * vector

Say you have a user on Netflix, represented as their ratings across 1,000 movies:

user = [5, 0, 3 ...] # 1000 numbers

Most of those 1,000 numbers mean nothing, but some could be reducible to a few underlying taste dimensions or how much they like genres.

So you multiply by a learned matrix (movies · 3)...

user · W = [0.92, 0.11, 0.87] # 3 numbers

and now this is a point in 3D taste space.

* matrix · vector = projection into a new space. The matrix is learned so that the new space is more useful than the original.

EIGENVECTORS

- most of the time, when a matrix transforms a vector the vector changes direction (rotated, stretched, flipped, ...).
- eigenvectors are different because when you apply a matrix to it, it doesn't change direction.
  i) it only gets scaled - stretched or shrunk on the same line it was already on.

example for eigenvectors use in ML

Back to Netflix, you have 5 users, each rated 3 movies:

	1	2	3
user1 =	5	1	7
user2 =	2	3	7
user3 =	3	5	8
...			

→ if you plot these 5 users in a 3D space, each is a point.

Then PCA recognizes the variation along one diagonal direction, which represents the first eigenvector.

SO... it took the 3 numbers → 1 number per user.
i) PCA found a label purley from variance in data.
ii) the eignevector revealed the structure that was there.

PCA (Principal Component Analysis) uses eignvectors + algorithm to find most important directions in the data and drop the unimportant.

Dot product = one number, scalar, measuring how much two vectors allign.

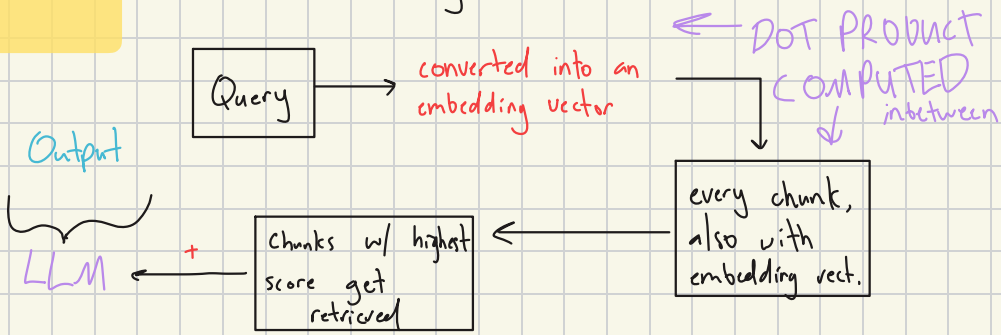
i) Large = similar direction
ii) Zero = no relation
iii) Negative = opposite

* close to cosine similarity, but includes magnitude unlike cosine similarity which only accounts for direction.

Dot Product in ML

HOW IT WORKS IN RAG:

-retrieval works by...



* this is the math underneath semantic search / FAISS at smallest scale.

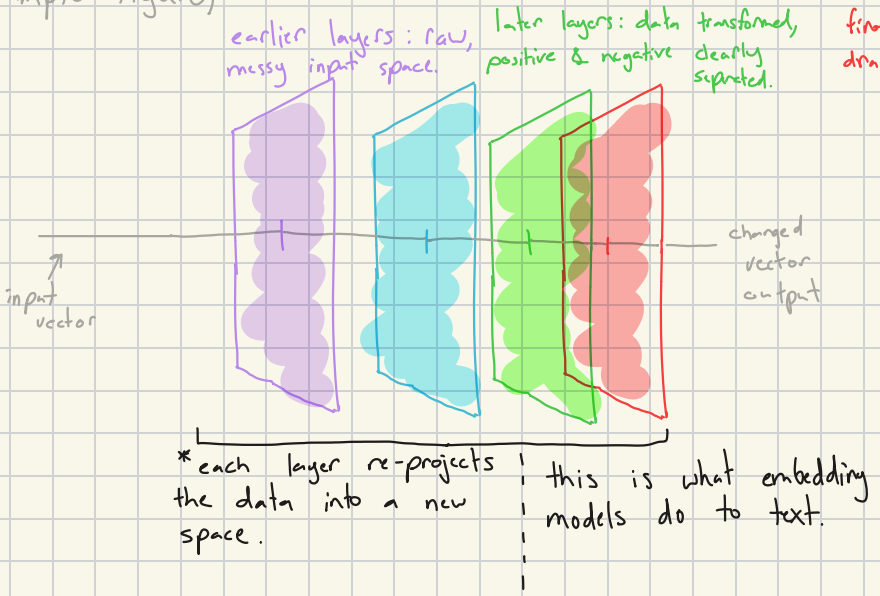
Orthogonal vectors with a dot product of zero carry complete independence of information. This means that in PCA, all principle components are orthogonal by design - each one captures variation where independence of direction = no redundant info.

SHORT SUMMARY...

vector = direction + magnitude.
matrix = learned projection into a new space.
eigenvector = direction of maximum variance, revealed by data.
dot product = allignment measurement between two vectors.
orthogonal = zero allignment and independent information

* now... neural networks. A neural network is a sequence of matrix multiplications that progressively reshape your data into a space where the answer is linearly seperable.

example figure)



SVD - Singular Value Decomposition

*breaking down a X by X matrix (any size) into 3 parts revealing structure and allowing multiplication of matrix regardless of dimensions.

formula: $A = U \Sigma V^T$ ← maxtrix (list of lists)

U: matrix (list of lists)

Σ: sigma is list of singular values

Rotate → Scale → Rotate

Low-rank approximation: keep top k singular values, discard the rest. This is how SVD compresses data.

Decomposition of $A = U\Sigma V^T$:

U - orthogonal matrix (no alignment + independent) whos columns are the left singular vectors or directions in the output space.

Σ - diagonal matrix of singular values (how much each direction scales + always non-negative).

V^T - orthogonal matrix whos columns are the right single vectors.

So geometrically, any matrix A is ...

rotation (V^T) → stretch (Σ) → another rotation (U)

Gradient & Gradient Descent

The problem:

you have a model with weights. Those weights produce a loss - some number telling you how wrong the model is. We need to adjust the weights to minimize the loss.

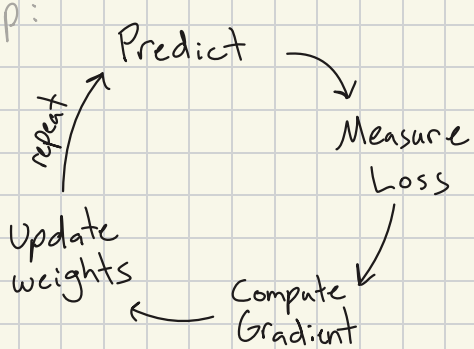
SO... which way do we adjust them?

A loss function is a single number that measures how wrong the model is. A gradient is then used to tell you how to adjust the weights.

$$w \leftarrow w - \eta \nabla L(w) \quad *$$

* new weight = old weight minus learning rate multiplied by gradient.

Training Loop:



Bayes' Theorem

* the core idea is about updating your belief when you get new evidence.

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

prior = belief before evidence.
posterior = belief after evidence.
likelihood = how probable an event is given assumption